Projektowanie i analiza algorytmów
Sprawozdanie
Miniprojekt 1

Filip Juchnowicz

Indeks 264176

03.04.2023

Spis treści

1 Zadanie 1 - Tablice dynamiczne. 4
- 1.1 Operacje na tablicy. . . . 4
  - 1.1.1 Wypełnianie tablicy losowymi wartościami . . . 4
  - 1.1.2 Wyświetlanie tablicy . . . 4
  - 1.1.3 Znajdź maksymalny element . . . 5
- 1.2 Zapis tablicy do pliku . . . 5
  - 1.2.1 tekstowego . . . 5
  - 1.2.2 binarnego . . . 6

2 Zadanie 2 - Funkcje rekurencyjne. 6
- 2.1 Matematyczne . . . 6
- 2.2 Palindrom . . . 7
- 2.3 Permutacje ciągu znaków . . . 8
  - 2.3.1 Generacja palindromów . . . 8
  - 2.3.2 Usuwanie duplikatów . . . 8

3 Struktury 9
- 3.1 Węzeł . . . 9
- 3.2 Lista jednokierunkowa . . . 9
  - 3.2.1 Dodawanie . . . 9
  - 3.2.2 Usuwanie . . . 10
  - 3.2.3 Wyświetlanie . . . 11
  - 3.2.4 Czyszczenie struktury . . . 12
- 3.3 Kolejka . . . 12
  - 3.3.1 Dodawanie . . . 12
  - 3.3.2 Usuwanie . . . 13
  - 3.3.3 Wyświetlanie . . . 13
  - 3.3.4 Czyszczenie kolejki . . . 14
- 3.4 Dequeue . . . 14
  - 3.4.1 Węzeł . . . 14
  - 3.4.2 Insert last . . . 14
  - 3.4.3 Shift, Pop . . . 15
  - 3.4.4 isEmpty . . . 16

Spis rysunków

1 Funkcja wypełniająca tablice wartościami z zakresu <0; x> . . . 4
2 Funkcja wyświetlające zawartość tablicy w terminalu. . . . 4
3 Funkcja znajdująca maksymalny element w tablicy. . . . 5
4 Zapis tablicy do pliku tekstowego. . . . 5
5 Zapis tablicy do pliku binarnego . . . 6
6 Rekurencyjna funkcja licząca potęgę. . . . 6
7 Rekurencyjna funkcja licząca średnią. . . . 7

8 Rekurencyjna funkcja sprawdzająca czy słowo jest palindromem. 7
9 Funkcja generujące permutacje ciągu znaków i zapisująca palindromy. 8
10 Funkcja usuwająca duplikowane palindromy. 8
11 Struktura węzła. 9
12 Klasa listy jednokierunkowej. 9
13 Metoda dodająca węzeł do listy. 10
14 Metoda usuwająca węzeł 11
15 Metoda wyświetlająca strukturę. 12
16 Metoda czyszcząca listę. 12
17 Metoda dodająca węzeł do kolejki. 13
18 Metoda usuwająca element 13
19 Węzeł dla dequeue. 14
20 Metoda dodająca element na końcu dequeue. 15
21 Metody usuwania elementów. 15
22 Metoda sprawdzający czy dequeue nie zawiera węzłów. 16

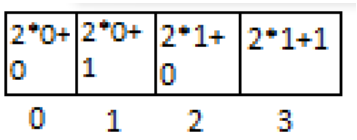
1 Zadanie 1 - Tablice dynamiczne.

Tablica jest tworzona poprzez przypisanie zmiennej jednego ciągu adresów w pamięci o podanym rozmiarze (np. 2x4 - 8 komórek).

1.1 Operacje na tablicy.

1.1.1 Wypełnianie tablicy losowymi wartościami

Funkcja odwołuje się do tablicy poprzez wskaźnik. Ponieważ tablica jest jednym dużym blokiem adresów odwołanie polega na wskazanie wiersza $(i * n)$, a następnie kolumny $(+j)$.



- Złożoność obliczeniowa: O(nm),
- Złożoność pamięciowa: O(1).

```
void fill_array(int* array, int x, int n, int m){
    srand( Seed: time( Time: NULL));
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            array[i * n + j] = rand() % x;
}
```

Rysunek 1: Funkcja wypełniająca tablice wartościami z zakresu <0; x>

1.1.2 Wyświetlanie tablicy

Wyświetlanie działa identycznie jak wypełnianie wartościami.

```
void print_array(int* array, int n, int m){
    for(int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++)
            cout << array[i * n + j] << " ";
        cout << endl;
    }
}
```

Rysunek 2: Funkcja wyświetlające zawartość tablicy w terminalu.

1.1.3 Znajdź maksymalny element

Funkcja jest kolejną modyfikacją funkcji wypełniającej tablice. Zawiera dodatkową zmienną (szukane max) oraz warunek sprawdzający czy aktualny element jest większy od zapisanego max.

```
int find_max(int* array, int n, int m){
    int max = -1;
    for(int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++)
            if(max < array[i * n + j])
                max = array[i * n + j];
    }

    return max;
}
```

Rysunek 3: Funkcja znajdująca maksymalny element w tablicy.

1.2 Zapis tablicy do pliku

1.2.1 tekstowego

Funkcje działają podobnie jak poprzednie. Liczby są pojedynczo wpisywane/sczytywane uwzględniając znak nowej linii na końcu.

```
void save_to_file(int* array, int n, int m){
    ofstream file( s "../txt_files/zapis.txt",  mode ios::out);

    if(file.is_open()){
        for(int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++)
                file << array[i * n + j] << " ";
            file << "\n";
        }
        file.close();
    }
    else
        cout<<"Error opening a file\n";
}

void read_from_file(int* array, int n , int m){
    ifstream file( s "../txt_files/zapis.txt",  mode ios::in);

    if(file.is_open()){
        for(int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++){
                int tmp;
                file >> tmp;
                array[i * n + j] = tmp;
            }
        }
        file.close();
    }
    else
        cout<<"Error opening a file\n";
}
```

Rysunek 4: Zapis tablicy do pliku tekstowego.

1.2.2 binarnego

Zapisywanie do pliku binarnego polega na wpisaniu danych na zadeklarowany blok o odpowiednim rozmiarze.



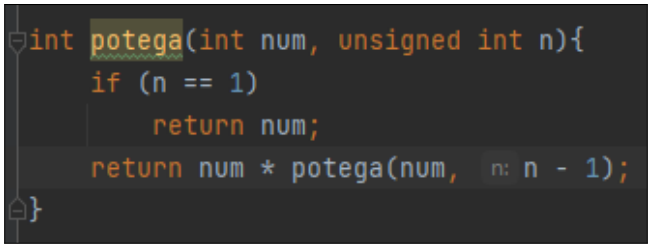
Rysunek 5: Zapis tablicy do pliku binarnego

2 Zadanie 2 - Funkcje rekurencyjne.

2.1 Matematyczne

Zadanie polegało na stworzeniu funkcji rekurencyjnych obliczających potęgę i silnię. Funkcja **potęga** rekurencyjnie tworzy iloczyn o długości wykładnika (np. $2^3 = 2 * 2 * 2$), zmniejszając go o jeden z każdym wywołaniem. Warunkiem wyjściowym z rekurencji jest osiągnięcie przez wykładnik wartości 1.

- Złożoność obliczeniowa: O(n),
- Złożoność pamięciowa: O(1).



Rysunek 6: Rekurencyjna funkcja licząca potęgę.

Funkcja do obliczenia silni działa w ten sam sposób. Tworzy malejący iloczyn ciągu, aż do osiągnięcia jedynki (np. $3! = 3*2*1$, gdzie 1 jest warunkiem wyjścia z rekurencji).

- Złożoność obliczeniowa: O(n),
- Złożoność pamięciowa: O(1).

```
int silnia(int x){
    if(x == 1)
        return 1;
    return x * silnia( x: x - 1);
}
```

Rysunek 7: Rekurencyjna funkcja licząca średnią.

2.2 Palindrom

Palindrom to słowo, które napisane od końca będzie takie samo. Funkcja działa poprzez odwoływanie się do zmiennej string po indeksach (jest traktowana jako tablica znaków). Kolejno sprawdzamy czy znaki są takie same, przesuwając "wskaźniki"bliżej środka wyrazu z każdą rekurencją. Jeśli w dowolnym momencie znaki będą inne, słowo nie jest palindromem i zwracany jest **fałsz**. Rekurencja kończy się, gdy lewy indeks będzie większy od prawego (wskaźniki "minęły"się, co oznacza że lewa i prawa część wyrazu została sprawdzona).

- Złożoność obliczeniowa: O(n),
- Złożoność pamięciowa: O(1).

```
bool jest_pal(int x, int y, string word){
    if(x >= y)
        return 1;
    if(word[x] != word[y])
        return 0;
    return jest_pal( x: x + 1,  y: y - 1, word);
}
```

Rysunek 8: Rekurencyjna funkcja sprawdzająca czy słowo jest palindromem.

2.3 Permutacje ciągu znaków

2.3.1 Generacja palindromów

Funkcja przyjmuje tablice (rozmiar oparty na maksymalnej ilości wygenerowanych permutacji), do której zapisuje palindromy.

- Złożoność obliczeniowa: O(n!),
- Złożoność pamięciowa: O(1).

```
int cnt = 0;
void permute(string *array, string word, unsigned short l, unsigned short r){
    if(l == r){
        if(jest_pal( x: 0,  y: word.size()-1, word)) {
            array[cnt] = word;
            cnt++;
        }
    }
    for(int i = l; i <= r; i++){
        swap( &: word[l],  &: word[i]);
        permute(array, word,  l: l+1, r);
        swap( &: word[l],  &: word[i]);
    }
}
```

Rysunek 9: Funkcja generujące permutacje ciągu znaków i zapisująca palindromy.

2.3.2 Usuwanie duplikatów

Funkcja zapisuje wyraz, a następnie sprawdza czy istnieją duplikaty (ignorując "puste"miejsca w tablicy). Jeśli powtórzenie wystąpi, to jest ono nadpisywane pustym stringiem.

- Złożoność obliczeniowa: O(n^2),
- Złożoność pamięciowa: O(1).

```
void usunDup(string *array){
    for(int i = 0; i < cnt; i++){
        if(array[i] == "")
            continue;
        string tmp = array[i];
        for(int j = 1; j < cnt; j++){
            if(i == j)
                break;
            string tmpCompare = array[j];
            if(tmp == tmpCompare)
                array[j] = "";
        }
    }
}
```

Rysunek 10: Funkcja usuwająca duplikowane palindromy.

3 Struktury

3.1 Węzeł

Każda z zaimplementowanych struktur opiera się na węzłach. Są to obiekty przechowujące wartość oraz wskaźnik na następny element.

```
class Node {
public:
    int val;
    Node* next;
    explicit Node(int data);
    Node();
};
```

Rysunek 11: Struktura węzła.

3.2 Lista jednokierunkowa

Lista jednokierunkowa jest prostą strukturą. Każdy element przechowuje wartość oraz wskaźnik na następny węzeł. Do listy odwołuje się przez "głowę"(czyli pierwszy element), a ostatni wskazuje na nic (nullptr).

```
class LinkedList {
private:
    Node* head;
public:
    explicit LinkedList(int data);
    void insert(int val);
    void purge();
    void delete_node(int offset);
    void print();
};
```

Rysunek 12: Klasa listy jednokierunkowej.

3.2.1 Dodawanie

Metoda dzieli się na dwa przypadki. Dla pustej listy ustawiana jest nowa głowa. W innym przypadku tworzony jest tymczasowy wskaźnik, który przesuwany jest aż wskaże na ostatni węzeł, gdzie następnie jest dodawany nowy element.

- Złożoność obliczeniowa: O(n),
- Złożoność pamięciowa: O(1).

```
void LinkedList::insert(int val){
    auto* new_node = new Node( data: val);

    if(head == nullptr){
        head = new_node;
        return;
    }

    Node* tmp = head;
    while(tmp->next != nullptr)
        tmp = tmp->next;

    tmp->next = new_node;
}
```

Rysunek 13: Metoda dodająca węzeł do listy.

3.2.2 Usuwanie

Metoda sprawdza kilka sytuacji. Jeśli lista jest pusta wyświetla adekwatny komunikat. Jeśli istnieje jeden element, trzeba go usunąć i ustawić głowę na nullptr. Gdy usuwany jest pierwszy element trzeba przestawić głowę na nowy węzeł. W ostatnim przypadku usuwany jest podany węzeł oraz przepina się elementy.

- Złożoność obliczeniowa: O(n),
- Złożoność pamięciowa: O(1).

```
void LinkedList::delete_node(int offset){
    Node* tmp1 = head;
    if(head == nullptr){
        cout<<"Lista jest pusta.\n";
        return;
    }

    if(head->next == nullptr){
        free( Memory: head);
        head = nullptr;
        return;
    }

    if(offset == 0){
        head = head->next;
        free( Memory: tmp1);
        return;
    }

    Node* tmp2;
    while(offset-- >= 1){
        tmp2 = tmp1;
        tmp1 = tmp1->next;
    }

    tmp2->next = tmp1->next;
    free( Memory: tmp1);
}
```

Rysunek 14: Metoda usuwająca węzeł

3.2.3 Wyświetlanie

Dla pustej listy metoda wyświetla adekwatny komunikat. W innym przypadku metoda przechodzi przez wszystkie węzły, wyświetlając wartości, aż zacznie wskazywać na nullptr (pusty węzeł).

- Złożoność obliczeniowa: O(n),
- Złożoność pamięciowa: O(1).

```
void LinkedList::print(){
    Node* tmp = head;

    if(head == nullptr){
        cout<<"Lista jest pusta.\n";
        return;
    }

    while(tmp != nullptr){
        cout<<tmp->val<<" ";
        tmp = tmp->next;
    }
}
```

Rysunek 15: Metoda wyświetlająca strukturę.

3.2.4 Czyszczenie struktury

Metoda zapisuję aktualną głowę listy, zmienia ją na następny element, następnie usuwa, powtarzając aż dojdzie do ostatniego węzła.

- Złożoność obliczeniowa: O(n),
- Złożoność pamięciowa: O(1).

```
void LinkedList::purge(){
    Node *tmp;
    while(head->next != nullptr){
        tmp = head;
        head = head->next;
        free( Memory: tmp);
    }
    tmp = head;
    head = nullptr;
    free( Memory: tmp);
}
```

Rysunek 16: Metoda czyszcząca listę.

3.3 Kolejka

Kolejka FIFO (first-in first-out) została zaimplementowana przez modyfikację listy jednokierunkowej. Jest uproszczeniem listy, ponieważ dla kolejki węzły zawsze dodajemy na końcu, a ściągamy z początku.

3.3.1 Dodawanie

Ponownie ostatni element nie jest zapamiętany, więc dla każdej operacji dodania metoda musi przejść przez wszystkie elementy, aby dodać nowy węzeł na końcu.

- Złożoność obliczeniowa: O(n),
- Złożoność pamięciowa: O(1).

```
void Queue::push(int val){
    auto* new_node = new Node( data: val);

    if(head == nullptr){
        head = new_node;
        return;
    }

    Node* tmp = head;
    while(tmp->next != nullptr)
        tmp = tmp->next;

    tmp->next = new_node;
}
```

Rysunek 17: Metoda dodająca węzeł do kolejki.

3.3.2 Usuwanie

Usuwany (i zwracany) jest pierwszy element, dlatego sama operacja jest bardzo szybka ponieważ do struktury odwołujemy się poprzez głowę (pierwszy element).

- Złożoność obliczeniowa: O(1),
- Złożoność pamięciowa: O(1).

```
int Queue::pop(){
    Node* tmp = head;
    int return_val = tmp->val;
    head = head->next;
    free( Memory: tmp);
    return return_val;
}
```

Rysunek 18: Metoda usuwająca element

3.3.3 Wyświetlanie

Metoda wyświetlająca kolejkę jest identyczna jak dla listy jedno kierunkowej.

3.3.4 Czyszczenie kolejki

Metoda czyszcząca kolejkę jest identyczna jak dla listy jedno kierunkowej.

3.4 Dequeue

Dequeue jest strukturą podobną do kolejki, lecz możemy się odwołać do pierwszego elementu (head) i ostatniego (tail) oraz wykonywać na nich operacje dodawania i usuwania węzłów. Zadanie polegało na wykorzystaniu dequeue do sprawdzenia czy podany wyraz jest palindromem. Omówione zostaną tylko funkcję wykorzystane w tej metodzie (insert, shift, pop, isEmpty).

3.4.1 Węzeł

```
class Node {
public:
    char val;
    Node *next;
    Node *prev;
    Node(char data);
};
```

Rysunek 19: Węzeł dla dequeue.

3.4.2 Insert last

Dla pustej struktury węzeł jest ustawiony jednocześnie dla głowy i ogona. Jeśli węzły już istnieją wstawianie operacja odbywa się przez odwołanie do ogona. Dzięki temu wystarczy stworzyć węzeł, przepiąć wskaźniki i ustawić go jako nowy ogon.

- Złożoność obliczeniowa: O(1),
- Złożoność pamięciowa: O(1).

```
void Dequeue::insertLast(char data) {
    if(head == nullptr && tail == nullptr){
        Node *new_node = new Node(data);
        head = new_node;
        tail = new_node;
        size++;
        return;
    }
    Node *newTail = new Node(data), **tmp = &tail;
    newTail->prev = tail;
    tail->next = newTail;
    *tmp = newTail;

    size++;
}
```

Rysunek 20: Metoda dodająca element na końcu dequeue.

3.4.3 Shift, Pop

Obydwie metody usuwające działają na tej samej zasadzie poprzez operacje na odpowiednim końcu struktury. Operacja polega na przepięciu krańcowego węzła na nowy i usunięcie starego węzła. Na końcu zwracana jest wartość usuniętego elementu.

```
char Dequeue::shift() {
    if(size == 1){
        return orphanRemove();
    }

    Node **tmp = &head, *newHead = head->next, *currHead = head;
    char return_val = head->val;

    newHead->prev = nullptr;
    *tmp = newHead;
    delete currHead;

    size--;

    return return_val;
}
```

(a) Usuwanie elementów z początku dequeue.

```
char Dequeue::pop() {
    if(size == 1){
        return orphanRemove();
    }

    Node **tmp = &tail, *newTail = tail->prev, *currTail = tail;
    char return_val = tail->val;

    newTail->next = nullptr;
    *tmp = newTail;
    delete currTail;

    size--;

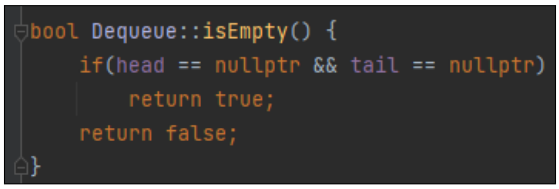
    return return_val;
}
```

(b) Usuwanie elementów z końca dequeue.

Rysunek 21: Metody usuwania elementów.

3.4.4 isEmpty

Żeby sprawdzić czy dequeue jest puste odwołujemy się do wskaźników na głowę i ogon. Przy pustej strukturze pierwszy element jest ustawiany na dwa wskaźniki, a dla usuwania ostatniego elementu (orphanRemove) metoda dba aby ustawić wskaźniki na nullptr.



Rysunek 22: Metoda sprawdzający czy dequeue nie zawiera węzłów.